

Introduction to Algorithms

CLRS 4th Edition

Comprehensive Algorithm Reference

Full explanations, pseudocode, step-by-step examples, and runtime analysis

Total algorithms covered: ~80 across all 35 chapters + appendices

Chapter 2: Getting Started

1. INSERTION-SORT(*A*, *n*)

Explanation:

Insertion sort works like sorting a hand of playing cards. We start with an empty left hand and pick cards one by one from the table. For each new card, we insert it into the correct position in our left hand, shifting larger cards to the right. It's an in-place, incremental sorting algorithm -- at step *i*, the subarray $A[1:i-1]$ is already sorted, and we insert $A[i]$ into its proper place.

Pseudocode:

```
INSERTION-SORT(A, n)
  for i = 2 to n:
    key = A[i]
    j = i - 1
    while j > 0 and A[j] > key:
      A[j+1] = A[j]
      j = j - 1
    A[j+1] = key
```

Step-by-Step Example:

$A = [5, 2, 4, 6, 1, 3]$, $n = 6$

$i=2$: $key=2$, $j=1$. $A[1]=5 > 2$ so shift 5 right. $A=[5,5,4,6,1,3]$, $j=0$. Place 2 at $j+1=1$. $A=[2,5,4,6,1,3]$

$i=3$: $key=4$, $j=2$. $A[2]=5 > 4$, shift 5 right. $A=[2,5,5,6,1,3]$, $j=1$. $A[1]=2 < 4$, stop. Place 4 at $j+1=2$. $A=[2,4,5,6,1,3]$

$i=4$: $key=6$, $j=3$. $A[3]=5 < 6$, no shift. $A=[2,4,5,6,1,3]$

$i=5$: $key=1$, $j=4$. Shift all: 6,5,4,2 right by 1. $A=[2,2,4,5,6,3]$, $j=0$. Place 1 at $0+1=1$. $A=[1,2,4,5,6,3]$

$i=6$: $key=3$, $j=5$. Shift 6,5,4 right. $A=[1,2,3,4,5,6]$ (sorted)

Worst: $O(n^2)$ | Best: $O(n)$ | Theta: $O(n^2)$

2. MERGE(*A*, *p*, *q*, *r*)

Explanation:

MERGE combines two sorted subarrays $A[p:q]$ and $A[q+1:r]$ into a single sorted subarray $A[p:r]$. It copies both halves into temporary arrays *L* and *R* with sentinel values (infinity) at the end, then repeatedly picks the smaller remaining element from the two arrays.

Pseudocode:

```
MERGE(A, p, q, r)
  n1 = q - p + 1
  n2 = r - q
  let L[1:n1+1] and R[1:n2+1] be arrays
  for i = 1 to n1: L[i] = A[p + i - 1]
  for j = 1 to n2: R[j] = A[q + j]
  L[n1+1] = R[n2+1] = INF
  i = j = 1
  for k = p to r:
    if L[i] <= R[j]:
      A[k] = L[i]; i++
    else:
```

```
A[k] = R[j]; j++
```

Step-by-Step Example:

Merge [3,12] and [7,9]: L=[3,12,INF], R=[7,9,INF]

k=0: L[1]=3 <= R[1]=7, A[0]=3, i=2

k=1: L[2]=12 > R[1]=7, A[1]=7, j=2

k=2: L[2]=12 > R[2]=9, A[2]=9, j=3

k=3: L[2]=12 < R[3]=INF, A[3]=12, i=3. Result: [3,7,9,12]

Worst: $O(n)$ | Best: $O(n)$ | Theta: $O(n)$

3. MERGE-SORT(A, p, r)

Explanation:

Merge sort is a classic divide-and-conquer sorting algorithm. It divides the array into two halves, recursively sorts each half, then merges the sorted halves. The base case is when the subarray has 0 or 1 elements (trivially sorted).

Pseudocode:

```
MERGE-SORT(A, p, r)
  if p < r:
    q = floor((p + r) / 2)
    MERGE-SORT(A, p, q)
    MERGE-SORT(A, q+1, r)
    MERGE(A, p, q, r)
```

Step-by-Step Example:

A = [12, 3, 7, 9, 14, 6, 11, 2], p=1, r=8

Divide: q=4 -> left[1..4], right[5..8]

Left: [12,3][7,9] -> [3,12] and [7,9] -> merge to [3,7,9,12]

Right: [14,6][11,2] -> [6,14] and [2,11] -> merge to [2,6,11,14]

Final merge: [3,7,9,12] + [2,6,11,14] -> [2,3,6,7,9,11,12,14]

Worst: $O(n \lg n)$ | Best: $O(n \lg n)$ | Theta: $O(n \lg n)$

Chapter 4: Divide-and-Conquer

4. FIND-MAXIMUM-SUBARRAY (Kadane's Algorithm)

Explanation:

Finds the contiguous subarray with the largest sum. Kadane's algorithm scans left to right, keeping track of the maximum sum ending at the current position and the global maximum. At each step, we decide whether to extend the current subarray or start a new one.

Pseudocode:

```
KADANE(A, n)
  max_ending_here = max_so_far = A[1]
  for i = 2 to n:
    max_ending_here = max(A[i], max_ending_here + A[i])
    max_so_far = max(max_so_far, max_ending_here)
  return max_so_far
```

Step-by-Step Example:

$A = [-2, 1, -3, 4, -1, 2, 1, -5, 4]$

$i=1$: $\text{max_ending_here} = -2$, $\text{max_so_far} = -2$

$i=2$: $\text{max_ending_here} = \max(1, -2+1 = -1) = 1$, $\text{max_so_far} = \max(-2, 1) = 1$

$i=3$: $\text{max_ending_here} = \max(-3, 1-3 = -2) = -2$, $\text{max_so_far} = \max(1, -2) = 1$

$i=4$: $\text{max_ending_here} = \max(4, -2+4 = 2) = 4$, $\text{max_so_far} = \max(1, 4) = 4$

$i=5$: $\text{max_ending_here} = \max(-1, 4-1 = 3) = 3$, $\text{max_so_far} = 4$

$i=6$: $\text{max_ending_here} = \max(2, 3+2 = 5) = 5$, $\text{max_so_far} = \max(4, 5) = 5$

$i=7$: $\text{max_ending_here} = \max(1, 5+1 = 6) = 6$, $\text{max_so_far} = \max(5, 6) = 6$

$i=8$: $\text{max_ending_here} = \max(-5, 6-5 = 1) = 1$, $\text{max_so_far} = 6$

$i=9$: $\text{max_ending_here} = \max(4, 1+4 = 5) = 5$, $\text{max_so_far} = 6$

Result: max sum = 6 (subarray [4, -1, 2, 1])

Worst: $O(n)$ | Best: $O(n)$ | Theta: $O(n)$

5. STRASSEN'S MATRIX MULTIPLICATION

Explanation:

Strassen's algorithm multiplies two $n \times n$ matrices using 7 recursive multiplications instead of 8. It partitions each matrix into 4 submatrices, computes 7 clever products (P1-P7) using additions/subtractions, then combines them. The key insight is that one multiplication can be saved by reusing intermediate results.

The 7 Products:

```
P1 = A11 * (B12 - B22)
P2 = (A11 + A12) * B22
P3 = (A21 + A22) * B11
P4 = A22 * (B21 - B11)
P5 = (A11 + A22) * (B11 + B22)
P6 = (A12 - A22) * (B21 + B22)
P7 = (A11 - A21) * (B11 + B12)
```

Combine:

$$C_{11} = P_5 + P_4 - P_2 + P_6$$

$$C_{12} = P_1 + P_2$$

$$C_{21} = P_3 + P_4$$

$$C_{22} = P_5 + P_1 - P_3 - P_7$$

Step-by-Step Example (2x2):

$$A = [[1,2],[3,4]], B = [[5,6],[7,8]]$$

$$P_1 = 1*(6-8) = -2, P_2 = (1+2)*8 = 24$$

$$P_3 = (3+4)*5 = 35, P_4 = 4*(7-5) = 8$$

$$P_5 = (1+4)*(5+8) = 65, P_6 = (2-4)*(7+8) = -30$$

$$P_7 = (1-3)*(5+6) = -22$$

$$C_{11} = 65+8-24+(-30) = 19, C_{12} = -2+24 = 22$$

$$C_{21} = 35+8 = 43, C_{22} = 65+(-2)-35-(-22) = 50$$

Result: [[19,22],[43,50]] (correct!)

Worst: $O(n^{\{2.807\}})$ | Best: $O(n^{\{2.807\}})$ | Theta: $O(n^{\{\lg 7\}})$

Chapter 6: Heapsort

6. PARENT / LEFT / RIGHT (Heap Index Helpers)

```
PARENT(i) = floor(i/2)
LEFT(i)   = 2i
RIGHT(i)  = 2i + 1
```

Worst: $O(1)$ | Best: $O(1)$ | Theta: $O(1)$

7. MAX-HEAPIFY(A, i)

Explanation:

Assumes the binary trees rooted at $LEFT(i)$ and $RIGHT(i)$ are max-heaps, but $A[i]$ may be smaller than its children. MAX-HEAPIFY lets $A[i]$ 'float down' by swapping it with its largest child until the max-heap property is restored.

Pseudocode:

```
MAX-HEAPIFY(A, i)
  l = LEFT(i); r = RIGHT(i)
  largest = i
  if l <= A.heap-size and A[l] > A[largest]:
    largest = l
  if r <= A.heap-size and A[r] > A[largest]:
    largest = r
  if largest != i:
    swap A[i] with A[largest]
    MAX-HEAPIFY(A, largest)
```

Step-by-Step Example:

$A = [27, 17, 3, 16, 13, 10, 1, 5, 7, 12, 4, 8, 9, 0]$, heap-size=14, call MAX-HEAPIFY(A, 3)
 $i=3$: $A[3]=3$, $LEFT(3)=6$ ($A[6]=10$), $RIGHT(3)=7$ ($A[7]=1$)
 largest=6 (value 10). Swap $A[3]$ with $A[6]$. $A=[27, 17, 10, 16, 13, 3, 1, 5, 7, 12, 4, 8, 9, 0]$
 Recurse on $i=6$: $A[6]=3$, $LEFT(6)=12$ ($A[12]=8$), $RIGHT(6)=13$ ($A[13]=9$)
 largest=13 (value 9). Swap $A[6]$ with $A[13]$. $A=[27, 17, 10, 16, 13, 9, 1, 5, 7, 12, 4, 8, 3, 0]$
 Recurse on $i=13$: leaf, stop.

Worst: $O(\lg n)$ | Best: $O(1)$ (leaf) | Theta: $O(\lg n)$

8. BUILD-MAX-HEAP(A, n)

Explanation:

Converts an unordered array into a max-heap in linear time by calling MAX-HEAPIFY on every non-leaf node, bottom-up. Leaves (indices $\text{floor}(n/2)+1$ to n) are trivially max-heaps.

Pseudocode:

```
BUILD-MAX-HEAP(A, n)
  A.heap-size = n
  for i = floor(n/2) down to 1:
    MAX-HEAPIFY(A, i)
```

Step-by-Step Example:

$A = [5, 3, 17, 10, 84, 19, 6, 22, 9]$, $n=9$. $\text{floor}(9/2)=4$.

$i=4$: MAX-HEAPIFY($A, 4$): $A[4]=10$, children 22, 9. Swap $10 \leftrightarrow 22$.

$i=3$: MAX-HEAPIFY($A, 3$): $A[3]=17$, children 19, 6. Swap $17 \leftrightarrow 19$.

$i=2$: MAX-HEAPIFY($A, 2$): $A[2]=3$, children 22, 84. Swap $3 \leftrightarrow 84$, recurse: swap $3 \leftrightarrow 22$.

$i=1$: MAX-HEAPIFY($A, 1$): $A[1]=5$, children 84, 19. Swap $5 \leftrightarrow 84$, recurse...

Worst: $O(n)$ | Best: $O(n)$ | Theta: $O(n)$

9. HEAPSORT(A, n)**Explanation:**

Heapsort sorts in-place by first building a max-heap, then repeatedly extracting the maximum (root) and placing it at the end of the array. After each extraction, we restore the heap property on the reduced heap.

Pseudocode:

```

HEAPSORT( $A, n$ )
  BUILD-MAX-HEAP( $A, n$ )
  for  $i = n$  down to 2:
    swap  $A[1]$  with  $A[i]$ 
     $A.\text{heap-size} = A.\text{heap-size} - 1$ 
    MAX-HEAPIFY( $A, 1$ )

```

Step-by-Step Example:

$A = [5, 13, 2, 25, 7, 17, 20, 8, 4]$, $n=9$

BUILD-MAX-HEAP: $A=[25, 13, 20, 8, 7, 17, 2, 5, 4]$

$i=9$: swap $25 \leftrightarrow 4$. heap-size=8. MAX-HEAPIFY $\rightarrow [20, 13, 17, 8, 7, 4, 2, 5 \mid 25]$

$i=8$: swap $20 \leftrightarrow 5$. heap-size=7. MAX-HEAPIFY $\rightarrow [17, 13, 5, 8, 7, 4, 2 \mid 20, 25]$

Continue: $[13, 8, 5, 2, 7, 4 \mid 17, 20, 25] \rightarrow [8, 7, 5, 2, 4 \mid 13, 17, 20, 25]$

$\rightarrow [7, 4, 5, 2 \mid 8, 13, 17, 20, 25] \rightarrow [5, 4, 2 \mid 7, 8, 13, 17, 20, 25]$

$\rightarrow [4, 2 \mid 5, 7, 8, 13, 17, 20, 25] \rightarrow [2, 4, 5, 7, 8, 13, 17, 20, 25]$ (sorted)

Worst: $O(n \lg n)$ | Best: $O(n \lg n)$ | Theta: $O(n \lg n)$

10. MAX-HEAP-EXTRACT-MAX(A)

```

MAX-HEAP-EXTRACT-MAX( $A$ )
   $\text{max} = A[1]$ 
   $A[1] = A[A.\text{heap-size}]$ 
   $A.\text{heap-size} -= 1$ 
  MAX-HEAPIFY( $A, 1$ )
  return  $\text{max}$ 

```

Worst: $O(\lg n)$ | Best: $O(\lg n)$ | Theta: $O(\lg n)$

11. MAX-HEAP-INCREASE-KEY(A, x, k)

```

MAX-HEAP-INCREASE-KEY( $A, x, k$ )
  if  $k < x.\text{key}$ : error
   $x.\text{key} = k$ 
  while  $i > 1$  and  $A[\text{PARENT}(i)].\text{key} < A[i].\text{key}$ :

```

```
swap A[i] with A[PARENT(i)]  
i = PARENT(i)
```

Worst: $O(\lg n)$ | Best: $O(1)$ | Theta: $O(\lg n)$

12. MAX-HEAP-INSERT(*A*, *x*, *n*)

```
MAX-HEAP-INSERT(A, x, n)  
  A.heap-size += 1  
  x.key = -INF  
  place x at end  
  MAX-HEAP-INCREASE-KEY(A, x, k)
```

Worst: $O(\lg n)$ | Best: $O(\lg n)$ | Theta: $O(\lg n)$

Chapter 7: Quicksort

13. PARTITION (Lomuto)

Explanation:

Partitions the subarray $A[p:r]$ around the pivot $A[r]$ (last element). Elements $\leq$ pivot go to the left, $>$ pivot go to the right. Returns the final index of the pivot.

Pseudocode:

```
PARTITION(A, p, r)
  x = A[r]                // pivot
  i = p - 1               // index of smaller element
  for j = p to r - 1:
    if A[j] <= x:
      i++
      swap A[i] with A[j]
  swap A[i+1] with A[r]    // place pivot
  return i + 1            // pivot index
```

Step-by-Step Example:

$A = [13, 19, 9, 5, 12, 8, 7, 4, 21, 2, 6, 11]$, $p=1$, $r=12$, $\text{pivot}=11$

$j=1$: $13 > 11$ no swap; $j=2$: $19 > 11$ no swap

$j=3$: $9 \leq 11 \rightarrow i=1$, swap $A[1]=13 \leftrightarrow A[3]=9 \rightarrow [9, 19, 13, 5, 12, 8, 7, 4, 21, 2, 6, 11]$

$j=4$: $5 \leq 11 \rightarrow i=2$, swap $19 \leftrightarrow 5 \rightarrow [9, 5, 13, 19, 12, 8, 7, 4, 21, 2, 6, 11]$

$j=5$: $12 > 11$; $j=6$: $8 \leq 11 \rightarrow i=3$, swap $13 \leftrightarrow 8 \rightarrow [9, 5, 8, 19, 12, 13, 7, 4, 21, 2, 6, 11]$

$j=7$: $7 \leq 11 \rightarrow i=4$, swap $19 \leftrightarrow 7 \rightarrow [9, 5, 8, 7, 12, 13, 19, 4, 21, 2, 6, 11]$

$j=8$: $4 \leq 11 \rightarrow i=5$, swap $12 \leftrightarrow 4 \rightarrow [9, 5, 8, 7, 4, 13, 19, 12, 21, 2, 6, 11]$

$j=9$: $21 > 11$; $j=10$: $2 \leq 11 \rightarrow i=6$, swap $13 \leftrightarrow 2 \rightarrow [9, 5, 8, 7, 4, 2, 19, 12, 21, 13, 6, 11]$

$j=11$: $6 \leq 11 \rightarrow i=7$, swap $19 \leftrightarrow 6 \rightarrow [9, 5, 8, 7, 4, 2, 6, 12, 21, 13, 19, 11]$

Final: swap $A[8]=12 \leftrightarrow A[12]=11 \rightarrow [9, 5, 8, 7, 4, 2, 6, 11, 21, 13, 19, 12]$, return 8

Worst: $O(n)$ | Best: $O(n)$ | Theta: $O(n)$

14. QUICKSORT(A, p, r)

Explanation:

Divide-and-conquer sorting. Partition the array around a pivot, then recursively sort the left and right partitions. No combine step is needed since the array is sorted in-place after both recursive calls.

Pseudocode:

```
QUICKSORT(A, p, r)
  if p < r:
    q = PARTITION(A, p, r)
    QUICKSORT(A, p, q-1)
    QUICKSORT(A, q+1, r)
```

Step-by-Step Example:

$A=[9, 5, 8, 7, 4, 2, 6, 11, 21, 13, 19, 12]$ after first partition at $q=8$

Left subarray [9,5,8,7,4,2,6] -> partition around 6

Right subarray [21,13,19,12] -> partition around 12

Continue recursively until all subarrays are size 0 or 1

Worst: $O(n)$ | Best: $O(n \lg n)$ | Theta: $O(n \lg n)$

15. RANDOMIZED-QUICKSORT(A, p, r)

Explanation:

Same as Quicksort but chooses a random pivot before partitioning. This ensures no single input (like sorted arrays) always causes worst-case behavior. The expected running time is $O(n \lg n)$ for any input.

Pseudocode:

```

RANDOMIZED-PARTITION( $A, p, r$ )
     $i = \text{RANDOM}(p, r)$ 
    swap  $A[r]$  with  $A[i]$ 
    return PARTITION( $A, p, r$ )

RANDOMIZED-QUICKSORT( $A, p, r$ )
    if  $p < r$ :
         $q = \text{RANDOMIZED-PARTITION}(A, p, r)$ 
        RANDOMIZED-QUICKSORT( $A, p, q-1$ )
        RANDOMIZED-QUICKSORT( $A, q+1, r$ )

```

Worst: $O(n)$ | Best: $O(n \lg n)$ | Theta: $O(n \lg n)$

16. HOARE-PARTITION(A, p, r)

Explanation:

The original partitioning method by C.A.R. Hoare. Uses two pointers from both ends moving toward each other. More efficient than Lomuto when elements are equal.

Pseudocode:

```

HOARE-PARTITION( $A, p, r$ )
     $x = A[p]$ 
     $i = p - 1$ 
     $j = r + 1$ 
    while TRUE:
        repeat  $j--$  until  $A[j] \leq x$ 
        repeat  $i++$  until  $A[i] \geq x$ 
        if  $i < j$ : swap  $A[i]$  with  $A[j]$ 
        else: return  $j$ 

```

Worst: $O(n)$ | Best: $O(n)$ | Theta: $O(n)$

Chapter 8: Sorting in Linear Time

17. COUNTING-SORT(A, n, k)

Explanation:

Counting sort assumes each input element is an integer in $[0, k]$. It counts how many elements have each value, then uses prefix sums to determine the positions of elements in the sorted output. It's stable (equal elements maintain relative order).

Pseudocode:

```
COUNTING-SORT( $A, n, k$ )
  let  $C[0:k] = 0$ 
  for  $j = 1$  to  $n$ :           // count frequencies
     $C[A[j]]++$ 
  for  $i = 1$  to  $k$ :           // prefix sums
     $C[i] = C[i] + C[i-1]$ 
  let  $B[1:n]$ 
  for  $j = n$  down to  $1$ :      // stable placement
     $B[C[A[j]]] = A[j]$ 
     $C[A[j]]--$ 
  return  $B$ 
```

Step-by-Step Example:

$A=[6,0,2,0,1,3,4,6,1,3,2]$, $n=11$, $k=6$

Counts: $C=[2,2,2,2,1,0,2]$ (indices 0-6)

Prefix: $C=[2,4,6,8,9,9,11]$

$j=11$ ($A=2$): $B[6]=2$, $C[2]=5$

$j=10$ ($A=3$): $B[8]=3$, $C[3]=7$

...continue reverse scan...

Final $B = [0,0,1,1,2,2,3,3,4,6,6]$

Worst: $O(n+k)$ | Best: $O(n+k)$ | Theta: $O(n+k)$

18. RADIX-SORT(A, n, d)

Explanation:

Radix sort sorts d -digit numbers by sorting on each digit position, from least significant to most significant. It must use a stable sort for each digit (usually counting sort).

Pseudocode:

```
RADIX-SORT( $A, n, d$ )
  for  $i = 1$  to  $d$ :           //  $i=1$  is LSD
    use stable sort to sort  $A$  on digit  $i$ 
```

Step-by-Step Example:

Numbers: $[329,457,657,839,436,720,355]$, $d=3$

After sort on digit 1 (units): $[720,355,436,457,657,329,839]$

After sort on digit 2 (tens): $[720,329,436,839,355,457,657]$

After sort on digit 3 (hundreds): [329,355,436,457,657,720,839] (sorted)

Worst: $O(d(n+k))$ | Best: $O(d(n+k))$ | Theta: $O(d(n+k))$

19. BUCKET-SORT(A, n)

Explanation:

Bucket sort assumes input is uniformly distributed in $[0, 1)$. It creates n buckets, distributes elements into buckets based on value, sorts each bucket (usually with insertion sort), then concatenates.

Pseudocode:

```

BUCKET-SORT(A, n)
  let B[0:n-1] be empty lists
  for i = 1 to n:
    insert A[i] into B[floor(n * A[i])]
  for i = 0 to n-1:
    sort B[i] with insertion sort
  concatenate B[0], B[1], ..., B[n-1]

```

Step-by-Step Example:

$A = [0.79, 0.13, 0.16, 0.64, 0.39, 0.20, 0.89, 0.53, 0.71, 0.42]$, $n=10$

Buckets: [0.13,0.16], [0.20], [0.39], [0.42], [0.53], [0.64], [0.71,0.79], [0.89]

Sort each bucket, concatenate -> [0.13,0.16,0.20,0.39,0.42,0.53,0.64,0.71,0.79,0.89]

Worst: $O(n)$ | Best: $O(n)$ | Theta: $O(n)$ avg, $O(n)$ worst

Chapter 9: Medians and Order Statistics

20. MINIMUM(A, n)

```

MINIMUM(A, n)
  min = A[1]
  for i = 2 to n:
    if min > A[i]:
      min = A[i]
  return min

```

Worst: $O(n)$ | Best: $O(n)$ | Theta: $O(n)$

21. Simultaneous MIN & MAX

Explanation:

Finds both min and max in at most $3 \cdot \text{floor}(n/2)$ comparisons by processing elements in pairs. For each pair, compare the two elements first, then compare the smaller to min and larger to max.

Pseudocode:

```

MIN-MAX(A, n)
  if n % 2 == 0: min = min(A[1], A[2]); max = max(A[1], A[2]); i=3
  else: min = max = A[1]; i=2
  while i < n:
    if A[i] < A[i+1]:
      min = min(min, A[i]); max = max(max, A[i+1])
    else:
      min = min(min, A[i+1]); max = max(max, A[i])
    i += 2
  return (min, max)

```

Step-by-Step Example:

$A=[3,7,1,9,2,5,8,4]$, $n=8$

Pair (3,7): compare $3 < 7$. min=3, max=7

Pair (1,9): compare $1 < 9$. min=min(3,1)=1, max=max(7,9)=9

Pair (2,5): compare $2 < 5$. min=min(1,2)=1, max=max(9,5)=9

Pair (8,4): compare $4 < 8$. min=min(1,4)=1, max=max(9,8)=9

Result: min=1, max=9 (4 comparisons vs 14 naive)

Worst: $O(n)$ | Best: $O(n)$ | Theta: $O(n)$

22. RANDOMIZED-SELECT(A, p, r, i)

Explanation:

Returns the i -th smallest element of $A[p:r]$ using randomized divide-and-conquer. Similar to Quicksort, it partitions around a random pivot, then recurses only on the side containing the i -th element.

Pseudocode:

```

RANDOMIZED-SELECT(A, p, r, i)
  if p == r: return A[p]

```

```

q = RANDOMIZED-PARTITION(A, p, r)
k = q - p + 1           // elements in left + pivot
if i == k: return A[q]   // pivot is answer
elif i < k: return RANDOMIZED-SELECT(A, p, q-1, i)
else: return RANDOMIZED-SELECT(A, q+1, r, i-k)

```

Step-by-Step Example:

Find 5th smallest in $A=[2,3,0,5,7,9,1,8,6,4]$, $n=10$

Suppose pivot=4 $\rightarrow$ partition: low=[2,3,0,1,4], high=[5,7,9,8,6]

$k = q - p + 1 = 5$. $i=5 == k \rightarrow$ return pivot=4

(If i were smaller: recurse left; if larger: recurse right with $i-k$)

Worst: $O(n)$ | Best: $O(n)$ | Theta: $O(n)$

23. SELECT (Deterministic, Median-of-Medians)

Explanation:

A deterministic $O(n)$ selection algorithm. It divides elements into groups of 5, finds the median of each group, recursively finds the median of the medians, partitions around that median, and recurses on the appropriate side.

Pseudocode:

```

SELECT(A, p, r, i)
// Divide into groups of 5, find median of each
// Recursively find median-of-medians x
q = PARTITION-AROUND(A, p, r, x)
k = q - p + 1
if i == k: return A[q]
elif i < k: return SELECT(A, p, q-1, i)
else: return SELECT(A, q+1, r, i-k)

```

Key Insight:

By using groups of 5, the median-of-medians pivot guarantees that at most $7n/10$ elements can be on the larger side of the partition. This gives the recurrence $T(n) \leq T(n/5) + T(7n/10) + O(n)$, which solves to $T(n) = O(n)$.

Worst: $O(n)$ | Best: $O(n)$ | Theta: $O(n)$

Chapter 10: Elementary Data Structures

24. STACK Operations (PUSH / POP)

```

STACK-EMPTY(S)
    return S.top == 0

PUSH(S, x)
    if S.top == S.size: error 'overflow'
    S.top++; S[S.top] = x

POP(S)
    if STACK-EMPTY(S): error 'underflow'
    S.top--; return S[S.top+1]

```

Worst: $O(1)$ | Best: $O(1)$ | Theta: $O(1)$

25. QUEUE Operations (ENQUEUE / DEQUEUE)

```

ENQUEUE(Q, x)
    Q[Q.tail] = x
    if Q.tail == Q.size: Q.tail = 1
    else: Q.tail++

DEQUEUE(Q)
    x = Q[Q.head]
    if Q.head == Q.size: Q.head = 1
    else: Q.head++
    return x

```

Worst: $O(1)$ | Best: $O(1)$ | Theta: $O(1)$

26. Doubly Linked List Operations

```

LIST-SEARCH(L, k)
    x = L.head
    while x != NIL and x.key != k:
        x = x.next
    return x

LIST-PREPEND(L, x)
    x.next = L.head; x.prev = NIL
    if L.head != NIL: L.head.prev = x
    L.head = x

LIST-INSERT(x, y) // insert x after y
    x.next = y.next; x.prev = y
    if y.next != NIL: y.next.prev = x
    y.next = x

LIST-DELETE(L, x)
    if x.prev != NIL: x.prev.next = x.next
    else: L.head = x.next
    if x.next != NIL: x.next.prev = x.prev

```

Worst: $O(n)$ search / $O(1)$ insert/delete | Best: | Theta:

Chapter 11: Hash Tables

27. CHAINED-HASH-INSERT(T, x)

```
LIST-PREPEND( $T[h(x.key)], x$ )
```

Worst: $O(1)$ | Best: $O(1)$ | Theta: $O(1)$

28. CHAINED-HASH-SEARCH(T, k)

```
return LIST-SEARCH( $T[h(k)], k$ )
```

Worst: $O(n)$ | Best: $O(1+\alpha)$ | Theta: $O(1+\alpha)$ avg

29. HASH-INSERT (Open Addressing)

```
HASH-INSERT( $T, k$ )
   $i = 0$ 
  repeat:
     $q = h(k, i)$ 
    if  $T[q] == \text{NIL}$ :
       $T[q] = k$ ; return  $q$ 
     $i++$ 
  until  $i == m$ 
  error 'overflow'
```

Worst: $O(m)$ | Best: $O(1/(1-\alpha))$ | Theta: $O(1/(1-\alpha))$ avg

30. HASH-SEARCH (Open Addressing)

```
HASH-SEARCH( $T, k$ )
   $i = 0$ 
  repeat:
     $q = h(k, i)$ 
    if  $T[q] == k$ : return  $q$ 
     $i++$ 
  until  $T[q] == \text{NIL}$  or  $i == m$ 
  return NIL
```

Worst: $O(m)$ | Best: $O(1)$ | Theta:

Chapter 12: Binary Search Trees

31. INORDER-TREE-WALK(*x*)

```
INORDER-TREE-WALK(x)
  if x != NIL:
    INORDER-TREE-WALK(x.left)
    print x.key
    INORDER-TREE-WALK(x.right)
```

Worst: $O(n)$ | Best: $O(n)$ | Theta: $O(n)$

32. TREE-SEARCH(*x*, *k*)

```
TREE-SEARCH(x, k)
  if x == NIL or k == x.key: return x
  if k < x.key: return TREE-SEARCH(x.left, k)
  else: return TREE-SEARCH(x.right, k)
```

Worst: $O(n)$ | Best: $O(1)$ | Theta: $O(h)$

33. TREE-MINIMUM / TREE-MAXIMUM

```
TREE-MINIMUM(x)
  while x.left != NIL: x = x.left
  return x

TREE-MAXIMUM(x)
  while x.right != NIL: x = x.right
  return x
```

Worst: $O(n)$ | Best: $O(1)$ | Theta: $O(h)$

34. TREE-SUCCESSOR(*x*)

Explanation:

Returns the next node in inorder traversal. If *x* has a right subtree, successor is the minimum of that subtree. Otherwise, go up until finding a node that is a left child.

```
TREE-SUCCESSOR(x)
  if x.right != NIL:
    return TREE-MINIMUM(x.right)
  y = x.p
  while y != NIL and x == y.right:
    x = y; y = y.p
  return y
```

Worst: $O(n)$ | Best: $O(1)$ | Theta: $O(h)$

35. TREE-INSERT(*T*, *z*)

```
TREE-INSERT(T, z)
  x = T.root; y = NIL
```

```
while x != NIL:
    y = x
    if z.key < x.key: x = x.left
    else: x = x.right
z.p = y
if y == NIL: T.root = z
elif z.key < y.key: y.left = z
else: y.right = z
```

Worst: $O(n)$ | Best: $O(1)$ | Theta: $O(h)$

Chapter 13: Red-Black Trees

36. LEFT-ROTATE(*T*, *x*)

Explanation:

Performs a left rotation on node *x* (*x.right* must not be NIL). The right child *y* becomes the new root of the subtree. *x* becomes *y*'s left child. *y*'s left child becomes *x*'s right child. BST order is preserved.

Pseudocode:

```
LEFT-ROTATE(T, x)
  y = x.right
  x.right = y.left
  if y.left != T.nil: y.left.p = x
  y.p = x.p
  if x.p == T.nil: T.root = y
  elif x == x.p.left: x.p.left = y
  else: x.p.right = y
  y.left = x
  x.p = y
```

Step-by-Step Example:

Before: *x*(7) with right child *y*(11), *y* has left child *beta*(9), right child *gamma*(13)

Tree: 7 -> (alpha, 11 -> (9, 13))

After LEFT-ROTATE:

11 -> (7 -> (alpha, 9), 13)

BST order $\alpha < 7 < 9 < 11 < 13$ preserved

Worst: $O(1)$ | Best: $O(1)$ | Theta: $O(1)$

37. RB-INSERT(*T*, *z*)

Pseudocode:

```
RB-INSERT(T, z)
  // Same as TREE-INSERT with T.nil sentinel
  z.left = z.right = T.nil
  z.color = RED
  RB-INSERT-FIXUP(T, z)
```

FIXUP Cases (*z*=red, *z.p*=red):

Case 1 (uncle red): recolor parent, uncle, grandparent; move *z* up 2.

Case 2 (uncle black, *z* is right child): left rotate on parent; falls to Case 3.

Case 3 (uncle black, *z* is left child): recolor parent=black, grandparent=red; right rotate on grandparent.

Step-by-Step Example:

Insert 38, 31, 12 into empty RB tree:

38: root (black)

31: left child of 38 (red)

12: left child of 31 (red) -> violation (two reds)

Uncle of 12 = NIL (black). Case 2/3: left-rotate 31, recolor
-> 31(black), left=12(red), right=38(red) [valid RB tree]

Worst: $O(\lg n)$ | Best: $O(\lg n)$ | Theta: $O(\lg n)$

38. RB-DELETE(T, z)

Explanation:

Deletes node z from the red-black tree. If a black node is removed, RB-DELETE-FIXUP restores properties. The fixup handles 4 cases where node x is 'doubly black' and the extra black is pushed up the tree.

Worst: $O(\lg n)$ | Best: $O(\lg n)$ | Theta: $O(\lg n)$

Chapter 14: Dynamic Programming

39. BOTTOM-UP-CUT-ROD(p, n)

Explanation:

Given a rod of length n and prices $p[i]$ for each length i , find the maximum revenue by cutting the rod into integer pieces. Uses DP: for each length j , try all possible first cuts i and use the previously computed optimal for the remaining $j-i$.

Pseudocode:

```
BOTTOM-UP-CUT-ROD(p, n)
  let r[0:n]
  r[0] = 0
  for j = 1 to n:
    q = -INF
    for i = 1 to j:
      q = max(q, p[i] + r[j-i])
    r[j] = q
  return r[n]
```

Step-by-Step Example:

prices $p=[1,5,8,9]$ for lengths 1,2,3,4

$j=1$: $q = \max(-\text{INF}, 1+r[0]=1) = 1$. $r[1]=1$

$j=2$: $q = \max(1+r[1]=2, 5+r[0]=5) = 5$. $r[2]=5$

$j=3$: $q = \max(1+r[2]=6, 5+r[1]=6, 8+r[0]=8) = 8$. $r[3]=8$

$j=4$: $q = \max(1+r[3]=9, 5+r[2]=10, 8+r[1]=9, 9+r[0]=9) = 10$. $r[4]=10$

Optimal: cut into 2+2, revenue = \$10

Worst: $O(n)$ | Best: $O(n)$ | Theta: $O(n)$

40. MATRIX-CHAIN-ORDER(p, n)

Explanation:

Given a chain of n matrices with dimensions $p[0] \times p[1]$, $p[1] \times p[2]$, ..., $p[n-1] \times p[n]$, find the optimal parenthesization to minimize scalar multiplications.

Pseudocode:

```
MATRIX-CHAIN-ORDER(p, n)
  let m[1:n, 1:n] and s[1:n-1, 2:n]
  for i = 1 to n: m[i,i] = 0
  for l = 2 to n: // chain length
    for i = 1 to n-l+1:
      j = i + l - 1
      m[i,j] = INF
      for k = i to j-1:
        q = m[i,k] + m[k+1,j] + p[i-1]*p[k]*p[j]
        if q < m[i,j]: m[i,j] = q; s[i,j] = k
  return m and s
```

Step-by-Step Example:

$A_1(10 \times 100)$, $A_2(100 \times 5)$, $A_3(5 \times 50)$, $n=3$

```

m[1,1]=0, m[2,2]=0, m[3,3]=0
l=2: i=1,j=2: k=1: q=0+0+10*100*5=5000. m[1,2]=5000
l=2: i=2,j=3: k=2: q=0+0+100*5*50=25000. m[2,3]=25000
l=3: i=1,j=3:
  k=1: q=0+25000+10*100*50=75000
  k=2: q=5000+0+10*5*50=7500 (optimal)
Optimal: ((A1A2)A3) = 7500 multiplications
Worst: O(n) | Best: O(n) | Theta: O(n)

```

41. LCS-LENGTH(X, Y)

Explanation:

Finds the length of the longest common subsequence of $X[1..m]$ and $Y[1..n]$. $c[i,j]$ is the LCS length of prefixes $X[1..i]$ and $Y[1..j]$.

Pseudocode:

```

LCS-LENGTH(X, Y)
  m = X.length; n = Y.length
  let c[0:m, 0:n]
  for i = 0 to m: c[i,0] = 0
  for j = 0 to n: c[0,j] = 0
  for i = 1 to m:
    for j = 1 to n:
      if X[i] == Y[j]:
        c[i,j] = c[i-1,j-1] + 1
      else:
        c[i,j] = max(c[i-1,j], c[i,j-1])
  return c

```

Step-by-Step Example:

$X=<A,B,C,B,D,A,B>$, $Y=<B,D,C,A,B,A>$

$c[1,1]$: $A \neq B$, $\max(0,0)=0$

$c[2,1]$: $B == B$, $c[1,0]+1=1$

...fill table...

$c[7,6]$: $\max(c[6,6], c[7,5]) = \max(4,4)=4$

LCS length = 4. One LCS = $<B, C, B, A>$

Worst: O(mn) | Best: O(mn) | Theta: O(mn)

42. OPTIMAL-BST(p, q, n)

Explanation:

Given keys $k_1..k_n$ with probabilities p_i and dummy keys $d_0..d_n$ with probabilities q_i , find the BST with minimum expected search cost. Uses DP on intervals $[i,j]$ of keys.

```

OPTIMAL-BST(p, q, n)
  for i = 1 to n+1:
    e[i,i-1] = q[i-1]; w[i,i-1] = q[i-1]
  for l = 1 to n:
    for i = 1 to n-l+1:
      j = i+l-1; w[i,j] = w[i,j-1] + p[j] + q[j]

```

```
e[i,j] = INF
for r = i to j:
    t = e[i,r-1] + e[r+1,j] + w[i,j]
    if t < e[i,j]: e[i,j] = t; root[i,j] = r
return e and root
```

Worst: $O(n)$ | Best: $O(n)$ | Theta: $O(n)$

Chapter 15: Greedy Algorithms

43. GREEDY-ACTIVITY-SELECTOR(*s*, *f*, *n*)

Explanation:

Selects the maximum number of non-overlapping activities. Activities are sorted by finish time. Always picks the activity with the earliest finish time that is compatible with the current selection.

Pseudocode:

```
GREEDY-ACTIVITY-SELECTOR(s, f, n)
  A = {a1}
  k = 1
  for m = 2 to n:
    if s[m] >= f[k]:
      A = A ∪ {am}
      k = m
  return A
```

Step-by-Step Example:

Activities (sorted by finish): *a*₁(1,4), *a*₂(3,5), *a*₃(0,6), *a*₄(5,7), *a*₅(3,9), *a*₆(5,9), *a*₇(6,10), *a*₈(8,11), *a*₉(8,12), *a*₁₀(2,14), *a*₁₁(12,16)

Pick *a*₁(1,4). *k*=1

m=2: *s*[2]=3 < *f*[1]=4, skip

m=3: *s*[3]=0 < *f*[1]=4, skip

m=4: *s*[4]=5 >= *f*[1]=4, pick *a*₄. *k*=4

m=5,6,7: skip (start before *f*[4]=7)

m=8: *s*[8]=8 >= *f*[4]=7, pick *a*₈. *k*=8

m=9,10: skip. *m*=11: *s*[11]=12 >= *f*[8]=11, pick *a*₁₁

Result: {*a*₁, *a*₄, *a*₈, *a*₁₁}

Worst: $O(n \lg n)$ | Best: $O(n)$ | Theta: $O(n \lg n)$ sorted

44. HUFFMAN(*C*)

Explanation:

Constructs an optimal prefix-free code for a set of characters with given frequencies. Repeatedly merges the two least-frequent characters into a new node whose frequency is their sum. The resulting tree gives the shortest possible encoded message length.

Pseudocode:

```
HUFFMAN(C)
  Q = C // min-priority queue keyed by frequency
  for i = 1 to |C|-1:
    z = new node
    x = EXTRACT-MIN(Q)
    y = EXTRACT-MIN(Q)
    z.left = x; z.right = y
    z.freq = x.freq + y.freq
    INSERT(Q, z)
```

```
return EXTRACT-MIN(Q)  // root
```

Step-by-Step Example:

Frequencies: a:45, b:13, c:12, d:16, e:9, f:5 (x1000)

Merge f(5) + e(9) -> 14: [a:45, d:16, b:13, c:12, (fe):14]

Merge c(12) + b(13) -> 25: [a:45, d:16, (fe):14, (cb):25]

Merge (fe)(14) + d(16) -> 30: [a:45, (cb):25, (fed):30]

Merge (cb)(25) + (fed)(30) -> 55: [a:45, (cbfed):55]

Merge a(45) + (cbfed)(55) -> 100: root

Codes: a=0, b=101, c=100, d=111, e=1101, f=1100

Total bits: 224,000 vs 300,000 fixed-length (25% savings)

Worst: $O(n \lg n)$ | Best: $O(n \lg n)$ | Theta: $O(n \lg n)$

Chapter 16: Amortized Analysis

45. MULTIPOP(S, k)

```
MULTIPOP(S, k)
  while not STACK-EMPTY(S) and k > 0:
    POP(S)
    k--
```

Amortized Analysis:

Over a sequence of n PUSH, POP, and MULTIPOP operations, each element is pushed at most once and popped at most once. Total cost is $O(n)$, so amortized cost per operation is $O(1)$.

Worst: $O(n)$ per seq | Best: $O(1)$ amortized | Theta: $O(1)$ amortized

46. INCREMENT (Binary Counter)

```
INCREMENT(A, k)
  i = 0
  while i < k and A[i] == 1:
    A[i] = 0
    i++
  if i < k: A[i] = 1
```

Amortized Analysis:

Bit $A[0]$ flips every time (n times), $A[1]$ flips every 2nd time ($n/2$), etc. Total flips = $n + n/2 + n/4 + \dots < 2n$. So amortized cost per INCREMENT = $O(1)$.

Worst: $O(n)$ per seq | Best: $O(1)$ amortized | Theta: $O(1)$ amortized

Chapter 17: Augmenting Data Structures

47. OS-SELECT(x , i)

Explanation:

Returns a pointer to the i -th smallest key in the order-statistic tree (red-black tree augmented with subtree sizes).

```
OS-SELECT( $x$ ,  $i$ )
   $r = x.\text{left.size} + 1$ 
  if  $i == r$ : return  $x$ 
  elif  $i < r$ : return OS-SELECT( $x.\text{left}$ ,  $i$ )
  else: return OS-SELECT( $x.\text{right}$ ,  $i - r$ )
```

Worst: $O(\lg n)$ | Best: $O(1)$ | Theta: $O(\lg n)$

48. OS-RANK(T , x)

```
OS-RANK( $T$ ,  $x$ )
   $r = x.\text{left.size} + 1$ 
   $y = x$ 
  while  $y \neq T.\text{root}$ :
    if  $y == y.p.\text{right}$ :
       $r += y.p.\text{left.size} + 1$ 
     $y = y.p$ 
  return  $r$ 
```

Worst: $O(\lg n)$ | Best: $O(\lg n)$ | Theta: $O(\lg n)$

49. INTERVAL-SEARCH(T , i)

Explanation:

Finds an interval in the interval tree that overlaps with query interval i . Uses the max attribute to safely prune subtrees.

```
INTERVAL-SEARCH( $T$ ,  $i$ )
   $x = T.\text{root}$ 
  while  $x \neq T.\text{nil}$  and  $i$  does not overlap  $x.\text{int}$ :
    if  $x.\text{left} \neq T.\text{nil}$  and  $x.\text{left.max} \geq i.\text{low}$ :
       $x = x.\text{left}$ 
    else:
       $x = x.\text{right}$ 
  return  $x$ 
```

Worst: $O(\lg n)$ | Best: $O(1)$ | Theta: $O(\lg n)$

Chapter 18: B-Trees

50. B-TREE-SEARCH(x, k)

```

B-TREE-SEARCH( $x, k$ )
   $i = 1$ 
  while  $i \leq x.n$  and  $k > x.key[i]$ :  $i++$ 
  if  $i \leq x.n$  and  $k == x.key[i]$ : return ( $x, i$ )
  if  $x.leaf$ : return NIL
  DISK-READ( $x.c[i]$ )
  return B-TREE-SEARCH( $x.c[i], k$ )

```

Worst: $O(\log_t n)$ | Best: $O(\log_t n)$ | Theta: $O(\log_t n)$

51. B-TREE-INSERT(T, k)

Explanation:

Inserts key k into a B-tree of minimum degree t . If the root is full, splits it (increasing height by 1). Then descends recursively, splitting any full node encountered before continuing.

```

B-TREE-INSERT( $T, k$ )
   $r = T.root$ 
  if  $r.n == 2t-1$ :
     $s = \text{new node}$ ;  $T.root = s$ 
     $s.leaf = \text{False}$ ;  $s.n = 0$ 
     $s.c[1] = r$ 
    B-TREE-SPLIT-CHILD( $s, 1$ )
    B-TREE-INSERT-NONFULL( $s, k$ )
  else: B-TREE-INSERT-NONFULL( $r, k$ )

```

Step-by-Step Example:

B-tree with $t=3$. Insert into root $[1,2,3,4,5]$ (full).

Split root at median 3: new root $[3]$ with children $[1,2]$ and $[4,5]$

Then insert k normally into appropriate child

Worst: $O(\log_t n)$ | Best: $O(\log_t n)$ | Theta: $O(\log_t n)$

52. B-TREE-DELETE(T, k)

Explanation:

Deletes key k while ensuring every node has at least $t-1$ keys. The algorithm restructures on the way down so the child being entered always has at least t keys, avoiding the need to backtrack.

Cases: (1) Leaf - remove directly. (2) Internal node with $\geq t$ child - swap with predecessor/successor, recurse. (3) Internal node with $t-1$ children - merge or borrow from sibling.

Worst: $O(\log_t n)$ | Best: $O(\log_t n)$ | Theta: $O(\log_t n)$

Chapter 19: Disjoint Sets

53. MAKE-SET(*x*)

```
MAKE-SET(x)
  x.p = x
  x.rank = 0
```

Worst: $O(1)$ | Best: $O(1)$ | Theta: $O(1)$

54. FIND-SET(*x*)

Explanation:

Returns the representative (root) of the set containing *x*. Uses path compression: after finding the root, it makes every node on the path point directly to the root.

```
FIND-SET(x)
  if x != x.p:
    x.p = FIND-SET(x.p)
  return x.p
```

Worst: $O(\alpha(n))$ | Best: $O(1)$ | Theta: $O(\alpha(n))$ amortized

55. UNION(*x*, *y*)

```
UNION(x, y)
  LINK(FIND-SET(x), FIND-SET(y))
```

```
LINK(x, y)
  if x.rank > y.rank: y.p = x
  else: x.p = y
  if x.rank == y.rank: y.rank++
```

Worst: $O(\alpha(n))$ | Best: $O(1)$ | Theta: $O(\alpha(n))$ amortized

Chapter 20: Elementary Graph Algorithms

56. BFS(*G*, *s*)

Explanation:

Breadth-First Search explores a graph level by level from source *s*. Uses a FIFO queue to process vertices. For unweighted graphs, it finds the shortest path distances (in number of edges) from *s* to all reachable vertices.

Pseudocode:

```

BFS(G, s)
  for each u in G.V:
    u.color = WHITE; u.d = INF; u.pi = NIL
  s.color = GRAY; s.d = 0; s.pi = NIL
  Q = {s}
  while Q != empty:
    u = DEQUEUE(Q)
    for each v in G.Adj[u]:
      if v.color == WHITE:
        v.color = GRAY; v.d = u.d + 1; v.pi = u
        ENQUEUE(Q, v)
    u.color = BLACK

```

Step-by-Step Example:

Graph: *s*-*a*, *s*-*b*, *a*-*b* (triangle), BFS from *s*

Initialize: *s.d*=0 (GRAY), *a.d*=INF, *b.d*=INF

Dequeue *s*: explore neighbors *a*,*b*

a: GRAY, *a.d*=1, enqueue

b: GRAY, *b.d*=1, enqueue

s: BLACK

Dequeue *a*: explore *b* (already GRAY), skip. *a*: BLACK

Dequeue *b*: no WHITE neighbors. *b*: BLACK

Result: *d*[*s*]=0, *d*[*a*]=1, *d*[*b*]=1

Worst: $O(V+E)$ | Best: $O(V+E)$ | Theta: $O(V+E)$

57. DFS(*G*)

Explanation:

Depth-First Search explores as deep as possible before backtracking. Uses recursion (or stack). Records discovery time (*d*) and finish time (*f*). Useful for topological sorting and SCC detection.

Pseudocode:

```

DFS(G)
  for each u in G.V: u.color = WHITE; u.pi = NIL
  time = 0
  for each u in G.V:
    if u.color == WHITE:
      DFS-VISIT(G, u)

```

```

DFS-VISIT(G, u)
  time++; u.d = time; u.color = GRAY
  for each v in G.Adj[u]:
    if v.color == WHITE:
      v.pi = u; DFS-VISIT(G, v)
  time++; u.f = time; u.color = BLACK

```

Step-by-Step Example:

Chain graph $u \rightarrow v \rightarrow w$

```

DFS-VISIT(u): u.d=1 (GRAY)
  DFS-VISIT(v): v.d=2 (GRAY), from v -> ...
    DFS-VISIT(w): w.d=3 (GRAY), w.f=4 (BLACK), return
  v.f=5 (BLACK), return
u.f=6 (BLACK), return
Timestamps: u(1,6), v(2,5), w(3,4)

```

Worst: $O(V+E)$ | Best: $O(V+E)$ | Theta: $O(V+E)$

58. TOPOLOGICAL-SORT(G)

Explanation:

Returns a linear ordering of a DAG's vertices such that for every edge (u,v) , u comes before v . Uses DFS finish times -- vertices are added to the front of a list as they finish.

```

TOPOLOGICAL-SORT(G)
  call DFS(G) to compute finish times
  as each vertex finishes, insert at front of linked list
  return linked list

```

Worst: $O(V+E)$ | Best: $O(V+E)$ | Theta: $O(V+E)$

59. STRONGLY-CONNECTED-COMPONENTS(G)

Explanation:

Finds strongly connected components by: (1) DFS on G to get finish times, (2) compute G^T (transpose), (3) DFS on G^T in decreasing finish time order. Each tree in the DFS forest is an SCC.

```

SCC(G)
  DFS(G) to compute finish times u.f
  compute  $G^T$  (reverse all edges)
  DFS( $G^T$ ), processing vertices in decreasing u.f order
  output each tree as an SCC

```

Worst: $O(V+E)$ | Best: $O(V+E)$ | Theta: $O(V+E)$

Chapter 21: Minimum Spanning Trees

60. KRUSKAL(G, w)

Explanation:

Kruskal's algorithm builds an MST by processing edges in increasing weight order. For each edge, if its endpoints are in different components (union-find), add it to the MST and merge the components.

Pseudocode:

```

KRUSKAL( $G, w$ )
   $A = \text{empty}$ 
  for each  $v$  in  $G.V$ : MAKE-SET( $v$ )
  sort edges by weight
  for each  $(u, v)$  in sorted order:
    if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ ):
       $A = A \cup \{(u, v)\}$ 
      UNION( $u, v$ )
  return  $A$ 

```

Step-by-Step Example:

$V = \{a, b, c\}$, edges: $(a, b) = 1$, $(b, c) = 2$, $(a, c) = 3$

Initially: $\{a\}, \{b\}, \{c\}$

Sorted: $(a, b) = 1$: FIND-SET(a) $\neq$ FIND-SET(b) $\rightarrow$ add, UNION $\rightarrow \{a, b\}, \{c\}$

$(b, c) = 2$: FIND-SET(b) $\neq$ FIND-SET(c) $\rightarrow$ add, UNION $\rightarrow \{a, b, c\}$

$(a, c) = 3$: same set, skip

MST weight = $1 + 2 = 3$

Worst: $O(E \lg V)$ | Best: $O(E \lg V)$ | Theta: $O(E \lg V)$

61. PRIM(G, w, r)

Explanation:

Prim's algorithm grows a single tree from root r . It maintains a priority queue of vertices not yet in the tree, keyed by the minimum weight edge connecting them to the tree. Always adds the vertex with the smallest key.

Pseudocode:

```

PRIM( $G, w, r$ )
  for each  $u$  in  $G.V$ :  $u.key = \text{INF}$ ;  $u.pi = \text{NIL}$ 
   $r.key = 0$ 
   $Q = G.V$ 
  while  $Q \neq \text{empty}$ :
     $u = \text{EXTRACT-MIN}(Q)$ 
    for each  $v$  in  $G.\text{Adj}[u]$ :
      if  $v$  in  $Q$  and  $w(u, v) < v.key$ :
         $v.pi = u$ 
         $v.key = w(u, v)$ 
        DECREASE-KEY( $Q, v, v.key$ )

```

Step-by-Step Example:

$V = \{a, b, c\}$, edges: $a-b(4)$, $a-c(2)$, $b-c(1)$, root= a

Init: $a.key=0$, $b.key=INF$, $c.key=INF$. $Q=\{a,b,c\}$

Extract a: relax $b=4$, $c=2$. $Q=\{b,c\}$

Extract c ($key=2$): relax b via c: $w(c,b)=1 < 4$, $b.key=1$, $b.pi=c$. $Q=\{b\}$

Extract b ($key=1$): done

MST: $a-c(2) + c-b(1) = \text{weight } 3$

Worst: $O(E \lg V)$ | Best: $O(E \lg V)$ | Theta: $O(E \lg V)$

Chapter 22: Single-Source Shortest Paths

62. BELLMAN-FORD(G, w, s)

Explanation:

Bellman-Ford finds shortest paths from s to all vertices, handling negative edge weights. It relaxes all edges $|V|-1$ times (the maximum number of edges in a simple path). Then it checks for negative-weight cycles.

Pseudocode:

```

BELLMAN-FORD( $G, w, s$ )
  INIT-SINGLE-SOURCE( $G, s$ )
  for  $i = 1$  to  $|V|-1$ :
    for each edge  $(u,v)$  in  $G.E$ :
      RELAX( $u, v, w$ )
  for each edge  $(u,v)$  in  $G.E$ :
    if  $v.d > u.d + w(u,v)$ : return FALSE // neg cycle
  return TRUE

RELAX( $u, v, w$ )
  if  $v.d > u.d + w(u,v)$ :
     $v.d = u.d + w(u,v)$ 
     $v.pi = u$ 

```

Step-by-Step Example:

4-vertex graph with edges: $s \rightarrow a(4)$, $s \rightarrow b(3)$, $a \rightarrow c(1)$, $b \rightarrow a(-2)$, $b \rightarrow c(2)$, $c \rightarrow t(3)$

Init: $s.d=0$, $a.d=b.d=c.d=t.d=INF$

Pass 1:

Relax $s \rightarrow a$: $a.d=4$. Relax $s \rightarrow b$: $b.d=3$

Relax $a \rightarrow c$: $c.d=5$. Relax $b \rightarrow a$: $a.d=\min(4, 3-2)=1$

Relax $b \rightarrow c$: $c.d=\min(5, 3+2)=5$. Relax $c \rightarrow t$: $t.d=8$

Pass 2: Relax $a \rightarrow c$: $c.d=\min(5, 1+1)=2$. Relax $c \rightarrow t$: $t.d=\min(8, 2+3)=5$

Pass 3: no changes. Pass 4: no changes.

Final: $s.d=0$, $a.d=1$, $b.d=3$, $c.d=2$, $t.d=5$

No negative cycle detected $\rightarrow$ return TRUE

Worst: $O(VE)$ | Best: $O(VE)$ | Theta: $O(VE)$

63. DAG-SHORTEST-PATHS(G, w, s)

Explanation:

For a DAG, we can find shortest paths in $O(V+E)$ by processing vertices in topological order. Relax each outgoing edge exactly once.

```

DAG-SHORTEST-PATHS( $G, w, s$ )
  topological sort  $G$ 
  INIT-SINGLE-SOURCE( $G, s$ )
  for each  $u$  in topological order:
    for each  $v$  in  $G.Adj[u]$ :
      RELAX( $u, v, w$ )

```

Worst: $O(V+E)$ | Best: $O(V+E)$ | Theta: $O(V+E)$

64. DIJKSTRA(G, w, s)

Explanation:

Dijkstra's algorithm finds shortest paths from s in graphs with nonnegative edge weights. It maintains a set S of settled vertices (whose shortest distance is known) and a priority queue of unprocessed vertices keyed by their current distance estimate.

Pseudocode:

```

DIJKSTRA( $G, w, s$ )
  INIT-SINGLE-SOURCE( $G, s$ )
   $S = \text{empty}$ 
   $Q = G.V$  // min-priority queue keyed by  $d$ 
  while  $Q \neq \text{empty}$ :
     $u = \text{EXTRACT-MIN}(Q)$ 
     $S = S \cup \{u\}$ 
    for each  $v$  in  $G.\text{Adj}[u]$ :
      RELAX( $u, v, w$ )

```

Step-by-Step Example:

Graph: $s \rightarrow a(1)$, $s \rightarrow b(4)$, $a \rightarrow b(2)$. $w \geq 0$.

Init: $s.d=0$, $a.d=\text{INF}$, $b.d=\text{INF}$. $Q=\{s,a,b\}$

Extract s ($d=0$): relax $a.d=1$, $b.d=4$. $S=\{s\}$

Extract a ($d=1$): relax $b.d=\min(4, 1+2=3)=3$. $S=\{s,a\}$

Extract b ($d=3$): done. $S=\{s,a,b\}$

Result: $s.d=0$, $a.d=1$, $b.d=3$. Path: $s \rightarrow a \rightarrow b$

Worst: $O(V)$ | Best: $O(V \lg V + E)$ | Theta: $O(V \lg V + E)$

Chapter 23: All-Pairs Shortest Paths

65. FLOYD-WARSHALL(W, n)

Explanation:

Floyd-Warshall solves APSP using DP over intermediate vertices. $d_{ij}^{(k)}$ is the shortest path from i to j using only vertices from $\{1, \dots, k\}$ as intermediates.

Pseudocode:

```
FLOYD-WARSHALL( $W, n$ )
   $D^{(0)} = W$  // direct edge weights (0 on diag, INF elsewhere)
  for  $k = 1$  to  $n$ :
    for  $i = 1$  to  $n$ :
      for  $j = 1$  to  $n$ :
         $d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$ 
  return  $D^{(n)}$ 
```

Step-by-Step Example:

3 vertices: edge weights: 1→2(3), 2→3(1), 1→3(8)

$D^{(0)}$: [0,3,8; INF,0,1; INF,INF,0]

$k=1$ (use 1 as intermediate): no changes (source 1 only)

$k=2$ (use {1,2}):

$d_{1,3} = \min(8, d_{1,2} + d_{2,3}) = \min(8, 3+1=4) = 4$

$k=3$ (use {1,2,3}): no changes

Result: [0,3,4; INF,0,1; INF,INF,0]

Worst: $O(V)$ | Best: $O(V)$ | Theta: $O(V)$

66. JOHNSON(G, w)

Explanation:

Johnson's algorithm efficiently solves APSP for sparse graphs. It reweights edges to be nonnegative (using Bellman-Ford), then runs Dijkstra from each vertex.

```
JOHNSON( $G, w$ )
  add vertex  $s$  with 0-weight edges to all  $v$ 
   $h = \text{Bellman-Ford}(G', w, s)$  // get  $h(v) = \delta(s, v)$ 
  for each  $(u, v)$ :  $w'(u, v) = w(u, v) + h(u) - h(v)$  // reweight, now  $\geq 0$ 
  for each  $u$  in  $G.V$ :
    Dijkstra( $G, w', u$ ) →  $d'_{uv}$ 
     $d_{uv} = d'_{uv} + h(v) - h(u)$ 
  return  $D$ 
```

Worst: $O(V \lg V + VE)$ | Best: $O(V \lg V + VE)$ | Theta: $O(V \lg V + VE)$

Chapter 24: Maximum Flow

67. FORD-FULKERSON(G, s, t)

Explanation:

Ford-Fulkerson finds the maximum flow from source s to sink t . While there is an augmenting path from s to t in the residual network, it sends as much flow as possible along that path. The residual network allows flow to be 'undone' via reverse edges.

Pseudocode:

```
FORD-FULKERSON( $G, s, t$ )
  for each  $(u, v)$  in  $G.E$ :  $f(u, v) = 0$ 
  while there is a path  $p$  from  $s$  to  $t$  in  $G_f$ :
     $c_f(p) = \min\{c_f(u, v) : (u, v) \text{ in } p\}$ 
    for each  $(u, v)$  in  $p$ :
      if  $(u, v)$  in  $E$ :  $f(u, v) += c_f(p)$ 
      else:  $f(v, u) -= c_f(p)$ 
  return  $f$ 
```

Step-by-Step Example:

Network: $s \rightarrow a(10)$, $s \rightarrow b(10)$, $a \rightarrow b(5)$, $a \rightarrow t(10)$, $b \rightarrow t(10)$

Path $s \rightarrow a \rightarrow t$: bottleneck=10. Send 10. $f(s, a)=10$, $f(a, t)=10$

Path $s \rightarrow b \rightarrow t$: bottleneck=10. Send 10. $f(s, b)=10$, $f(b, t)=10$

Residual: reverse flow on $s \rightarrow a$, $a \rightarrow t$, $s \rightarrow b$, $b \rightarrow t$

Path $s \rightarrow a \rightarrow b \rightarrow t$: bottleneck=5. Send 5.

$f(s, a)=15$, $f(a, b)=5$, $f(b, t)=15$

No more augmenting paths. Max flow = 25

Worst: $O(E|f^*|)$ | Best: $O(E|f^*|)$ | Theta: $O(E|f^*|)$

68. EDMONDS-KARP(G, s, t)

Explanation:

A variant of Ford-Fulkerson that always uses BFS to find the shortest augmenting path (minimum number of edges). This guarantees $O(VE^2)$ running time.

```
EDMONDS-KARP( $G, s, t$ )
  Same as Ford-Fulkerson, but find augmenting path using BFS
```

Worst: $O(VE)$ | Best: $O(VE)$ | Theta: $O(VE)$

Chapter 25: Matchings in Bipartite Graphs

69. GALE-SHAPLEY (Stable Matching)

Explanation:

Finds a stable matching between two equally-sized sets. Each person has a ranked preference list. While there is a free woman, she proposes to the highest-ranked man she hasn't proposed to yet. If he's free, they become engaged. If he prefers her to his current fiancée, she replaces her.

Pseudocode:

```
GALE-SHAPLEY(n)
  all men and women are free
  while exists free woman w:
    m = w's highest-ranked man not yet proposed to
    if m is free: engage (w, m)
    else if m prefers w to current fiancée w':
      break engagement (w', m); engage (w, m)
    else: w remains free, m rejects
  return engaged pairs
```

Worst: $O(n)$ | Best: $O(n)$ | Theta: $O(n)$

Chapter 28: Matrix Operations

70. LUP-SOLVE (Forward + Back Substitution)

Explanation:

Solves $Ax = b$ given LUP decomposition $PA = LU$. First solves $Ly = Pb$ (forward substitution), then solves $Ux = y$ (back substitution).

```
LUP-SOLVE(L, U, pi, b)
  n = L.rows
  // Forward substitution: Ly = Pb
  for i = 1 to n:
    y[i] = b[pi[i]] - sum_{j=1}^{i-1} L[i,j]*y[j]
  // Back substitution: Ux = y
  for i = n down to 1:
    x[i] = (y[i] - sum_{j=i+1}^n U[i,j]*x[j]) / U[i,i]
  return x
```

Worst: $O(n)$ | Best: $O(n)$ | Theta: $O(n)$

71. LU-DECOMPOSITION(A)

```
LU-DECOMPOSITION(A)
  n = A.rows
  for k = 1 to n:
    for i = k+1 to n:
      A[i,k] = A[i,k] / A[k,k]    // multiplier -> L
    for i = k+1 to n:
      for j = k+1 to n:
        A[i,j] = A[i,j] - A[i,k]*A[k,j]  // Schur complement -> U
  // L is unit lower-triangular, U is upper-triangular
```

Worst: $O(n)$ | Best: $O(n)$ | Theta: $O(n)$

Chapter 30: Polynomials and the FFT

72. FFT (Cooley-Tukey)

Explanation:

The Fast Fourier Transform computes the DFT (polynomial evaluation at n th roots of unity) in $O(n \lg n)$ time. It recursively splits the polynomial into even and odd terms, computes FFT on each half, then combines using the butterfly operation.

Pseudocode:

```

FFT(a, n)
  // a is coefficient vector [a0, a1, ..., an-1], n is power of 2
  if n == 1: return a
  omega_n = e{2*pi*i/n}
  a_even = FFT(a[even indices], n/2)
  a_odd = FFT(a[odd indices], n/2)
  for k = 0 to n/2 - 1:
    omega = exp(2*pi*i*k/n)
    y[k] = a_even[k] + omega * a_odd[k]
    y[k + n/2] = a_even[k] - omega * a_odd[k]
  return y

```

Step-by-Step Example:

Multiply $(1 + 2x) * (3 + 4x)$:

Pad to $n=4$: $A=[1,2,0,0]$, $B=[3,4,0,0]$

FFT(A): evaluate at $\omega^0.. \omega^3$

FFT(B): evaluate at same points

Pointwise multiply: $C[k] = A[k] * B[k]$ for $k=0..3$

Inverse FFT(C) $\rightarrow [3,10,8,0]$

Result: $3 + 10x + 8x^2$

Worst: $O(n \lg n)$ | Best: $O(n \lg n)$ | Theta: $O(n \lg n)$

Chapter 31: Number-Theoretic Algorithms

73. EUCLID(a, b)

Explanation:

Euclid's algorithm computes the GCD of two integers using the recurrence $\text{gcd}(a,b) = \text{gcd}(b, a \bmod b)$. It terminates when $b = 0$.

```
EUCLID(a, b)
  if b == 0: return a
  else: return EUCLID(b, a mod b)
```

Step-by-Step Example:

EUCLID(99, 78):

```
99 mod 78 = 21 -> EUCLID(78,21)
78 mod 21 = 15 -> EUCLID(21,15)
21 mod 15 = 6 -> EUCLID(15,6)
15 mod 6 = 3 -> EUCLID(6,3)
6 mod 3 = 0 -> EUCLID(3,0) -> return 3
```

Worst: $O(\lg b)$ | Best: $O(\lg b)$ | Theta: $O(\lg b)$

74. EXTENDED-EUCLID(a, b)

Explanation:

Computes integers x, y such that $ax + by = \text{gcd}(a,b)$. Uses the extended Euclidean algorithm.

```
EXTENDED-EUCLID(a, b)
  if b == 0: return (a, 1, 0)
  (d', x', y') = EXTENDED-EUCLID(b, a mod b)
  return (d', y', x' - floor(a/b) * y')
```

Step-by-Step Example:

EXTENDED-EUCLID(99,78):

```
(a,b) = (99,78): q=1. Recurse on (78,21)
(a,b) = (78,21): q=3. Recurse on (21,15)
(a,b) = (21,15): q=1. Recurse on (15,6)
(a,b) = (15,6): q=2. Recurse on (6,3)
(a,b) = (6,3): q=2. Recurse on (3,0)
(3,0): return (3,1,0)
Back to (6,3): return (3, 0, 1-2*0=1)
Back to (15,6): return (3, 1, 0-2*1=-2)
Back to (21,15): return (3, -2, 1-1*(-2)=3)
Back to (78,21): return (3, 3, -2-3*3=-11)
Back to (99,78): return (3, -11, 3-1*(-11)=14)
Check: 99*(-11) + 78*14 = -1089 + 1092 = 3
```

Worst: $O(\lg b)$ | Best: $O(\lg b)$ | Theta: $O(\lg b)$

75. MODULAR-EXPONENTIATION (Repeated Squaring)

```

MODULAR-EXPONENTIATION(a, b, n)
// returns  $a^b \bmod n$ 
c = 0; d = 1
binary = binary representation of b
for each bit in binary (MSB to LSB):
    c = 2*c
    d = (d*d) mod n
    if bit == 1:
        c = c + 1
        d = (d * a) mod n
return d

```

Step-by-Step Example:

Compute $7^{13} \bmod 11$:

$b=13$ in binary: 1101

Process bits MSB to LSB:

bit=1: $d=(1^2 \cdot 7) \% 11 = 7$

bit=1: $d=(7^2 \cdot 7) \% 11 = (49 \cdot 7) \% 11 = 343 \% 11 = 2$

bit=0: $d=(2^2) \% 11 = 4$

bit=1: $d=(4^2 \cdot 7) \% 11 = (16 \cdot 7) \% 11 = 112 \% 11 = 2$

Result: $7^{13} \bmod 11 = 2$

Worst: $O(\lg b)$ | Best: $O(\lg b)$ | Theta: $O(\lg b)$

Chapter 32: String Matching

76. NAIVE STRING MATCHER(T, P)

```

NAIVE-STRING-MATCHER( $T, P$ )
   $n = T.length; m = P.length$ 
  for  $s = 0$  to  $n-m$ :
    if  $P[1:m] == T[s+1:s+m]$ :
      print 'match at shift',  $s$ 

```

Worst: $O((n-m+1)m)$ | Best: $O(n)$ | Theta: $O((n-m+1)m)$

77. RABIN-KARP(T, P, d, q)

Explanation:

Uses a rolling hash to compare the pattern with each window of the text. If the hash values match, it checks for equality (to handle spurious hits). The rolling hash updates in $O(1)$ per shift.

```

RABIN-KARP( $T, P, d, q$ )
   $n = T.length; m = P.length$ 
   $h = d^{m-1} \bmod q$ 
   $p = 0; t = 0$ 
  for  $i = 1$  to  $m$ :           // precompute hashes
     $p = (d * p + P[i]) \bmod q$ 
     $t = (d * t + T[i]) \bmod q$ 
  for  $s = 0$  to  $n-m$ :
    if  $p == t$ :
      if  $P[1:m] == T[s+1:s+m]$ : match at  $s$ 
    if  $s < n-m$ :
       $t = (d * (t - T[s+1] * h) + T[s+m+1]) \bmod q$ 
      if  $t < 0$ :  $t += q$ 

```

Worst: $O((n-m+1)m)$ | Best: $O(n+m)$ | Theta: $O(n+m)$ expected

78. KMP (KNUTH-MORRIS-PRATT)

Explanation:

KMP achieves $O(n)$ matching time by preprocessing the pattern to compute a prefix function $\pi[q] = \text{length of longest proper prefix of } P[1:q] \text{ that is also a suffix}$. When a mismatch occurs, it uses π to skip ahead without re-scanning the text.

Prefix Function Computation:

```

COMPUTE-PREFIX( $P, m$ )
   $\pi[1] = 0; k = 0$ 
  for  $q = 2$  to  $m$ :
    while  $k > 0$  and  $P[k+1] != P[q]$ :  $k = \pi[k]$ 
    if  $P[k+1] == P[q]$ :  $k++$ 
     $\pi[q] = k$ 
  return  $\pi$ 

```

Matching:

```

KMP-MATCHER( $T, P$ )
   $n = T.length; m = P.length$ 

```

```

pi = COMPUTE-PREFIX(P, m)
q = 0
for i = 1 to n:
    while q > 0 and P[q+1] != T[i]: q = pi[q]
    if P[q+1] == T[i]: q++
    if q == m:
        print 'match at', i-m
        q = pi[q]

```

Step-by-Step Example:

P=ababaca, T=bacbababaabcbab

pi = [0,0,1,2,3,0,1] for q=1..7

Scan T:

i=1: T=b, P[a]!=b, q=0

i=2: T=a, P[1]=a, q=1

i=3: T=c, P[2]=b!=c, q=pi[1]=0. P[1]=a!=c, q=0

...continues with KMP avoiding re-scan...

Worst: $O(n)$ | Best: $O(n)$ | Theta: $O(n+m)$

Chapter 33: Machine-Learning Algorithms

79. LLOYD'S PROCEDURE (k-means)

Explanation:

Partitions n points into k clusters. Alternates between assigning each point to the nearest centroid and recomputing centroids as the mean of points in each cluster. Converges to a local minimum of the sum of squared distances.

```
K-MEANS( $X$ ,  $k$ )
  choose  $k$  initial centers randomly
  repeat:
    assign each point to nearest center
    recompute each center = mean of its points
  until no change
```

Worst: $O(T \cdot d \cdot k \cdot n)$ | Best: $O(T \cdot d \cdot k \cdot n)$ | Theta: $O(T \cdot d \cdot k \cdot n)$

80. WEIGHTED-MAJORITY (Multiplicative Weights)

Explanation:

An online learning algorithm that maintains weights for n experts. Each round, it predicts based on weighted majority vote, then decreases the weights of incorrect experts by factor $(1-\gamma)$.

```
WEIGHTED-MAJORITY( $\gamma$ )
  initialize all weights = 1
  for each event:
     $W_1$  = sum of weights predicting 1
     $W_0$  = sum of weights predicting 0
    predict 1 if  $W_1 \geq W_0$ , else 0
    multiply each incorrect expert's weight by  $(1-\gamma)$ 
```

Worst: $O(n \cdot T)$ | Best: $O(n \cdot T)$ | Theta: $O(n \cdot T)$

81. GRADIENT DESCENT

Explanation:

Minimizes a convex function f by iteratively moving in the direction of the negative gradient. The step size γ controls how far we move each step.

```
GRADIENT-DESCENT( $f$ ,  $T$ ,  $\gamma$ )
   $x = x_0$  (initial)
  for  $t = 0$  to  $T-1$ :
     $x = x - \gamma \cdot \text{gradient}(f, x)$ 
  return average of all  $x(t)$ 
```

Worst: $O(T \cdot \text{eval})$ | Best: $O(T \cdot \text{eval})$ | Theta: $O(T \cdot \text{eval})$

Chapter 35: Approximation Algorithms

82. APPROX-VERTEX-COVER(G)

Explanation:

A 2-approximation for vertex cover. Repeatedly picks an uncovered edge and adds both its endpoints to the cover. Since at least one endpoint of each picked edge is in any optimal cover, we're within factor 2.

```
APPROX-VERTEX-COVER(G)
  C = empty
  E' = G.E
  while E' != empty:
    pick any (u,v) in E'
    C = C U {u, v}
    remove from E' all edges incident to u or v
  return C
```

Step-by-Step Example:

Star graph: center a connected to b,c,d,e

Pick edge (a,b): add {a,b}. Remove edges (a,c),(a,d),(a,e) → E' empty

Cover: {a,b}. Size=2. Optimal cover: {a}. Ratio=2

Worst: $O(V+E)$ | Best: $O(V+E)$ | Theta: $O(V+E)$

83. APPROX-TSP-TOUR(G)

Explanation:

A 2-approximation for the metric TSP (satisfying triangle inequality). Computes an MST, then returns a preorder walk of the MST (which forms a Hamiltonian cycle after shortcutting).

```
APPROX-TSP-TOUR(G)
  T = MST(G) // e.g., Prim's
  H = preorder walk of T
  return H
```

Worst: $O(V)$ | Best: $O(V)$ | Theta: $O(V)$

84. GREEDY-SET-COVER(X, F)

Explanation:

An $O(\ln n)$ -approximation for set cover. At each step, picks the set that covers the most remaining uncovered elements.

```
GREEDY-SET-COVER(X, F)
  U = X (uncovered elements)
  C = empty
  while U != empty:
    pick S in F that maximizes |S intersect U|
    U = U - S
    C = C U {S}
  return C
```

Worst: $O(n \lg n)$ | Best: $O(n \lg n)$ | Theta: $O(n \lg n)$

85. APPROX-SUBSET-SUM (FPTAS)

Explanation:

A fully polynomial-time approximation scheme for subset sum. Trims the DP list to keep only representative sums, achieving $(1+\epsilon)$ -approximation in time polynomial in n and $1/\epsilon$.

```
APPROX-SUBSET-SUM(S, t, epsilon)
  L = {0}
  for each x in S:
    L = L U (L + x)
    L = TRIM(L, epsilon/(2*n))
  remove all elements > t
  return max(L)
```

Worst: $O(n/\epsilon)$ | Best: $O(n/\epsilon)$ | Theta: $O(n/\epsilon)$